

Honeywell Intake Scanning App

JIB 4338

Luke Hammond, Hoyeon Kang, Christian Chin, Barnabas Li, Nhat Nguyen

Client: Honeywell

Repository: <https://github.com/LukGH4/JIB-4338-HISA>

Table of Contents

<i>Table of Figures</i>	3
<i>Terminology</i>	4
<i>Introduction</i>	5
Background	5
Document Summary	5
<i>System Architecture</i>	6
Introduction.....	6
Rationale	6
Static System Architecture	7
Dynamic System Architecture	8
<i>Component Design</i>	10
Introduction.....	10
Static.....	10
Dynamic	12
<i>Data Design</i>	14
Introduction.....	14
Database Use	14
File Use	15
Data Exchange.....	16
<i>UI Design</i>	17
Introduction.....	17
Descriptions	17
<i>Appendix A</i>	23
Team Information	23

Table of Figures

Figure 1 Static System Architecture	7
Figure 2 Dynamic System Architecture.....	8
Figure 3 Component Design Class Diagram.....	11
Figure 4 Component Design Dynamic Diagram	13
Figure 5 Document Database Diagram	14
Figure 6 Document Database Example.....	15
Figure 7 Home Screen (Employee).....	17
Figure 8 Home Screen (Manager).....	17
Figure 9 Scan Screen (Employee)	18
Figure 10 Scan List.....	19
Figure 11 Scan List Bad Part Example	19
Figure 12 Scan Screen (Employee)	20
Figure 13 Part Statistics Example (Employee)	20
Figure 14 Employee Screen.....	21
Figure 15 Employee Data Example	21
Figure 16 Stats Screen (Manager)	22
Figure 17 Part Stats Comparison Example	22

Terminology

API (Application Programming Interface): A set of rules and protocols that allow different software applications to communicate with each other.

Authentication: The process of verifying a user's identity, typically through credentials like usernames and passwords or authentication tokens to ensure secure access.

Back-end: The part of an application that runs on a server, handling data storage, business logic, and communication between the front-end and databases or external services.

Firebase Authentication: A user authentication service from Google that allows for login and identity management for apps.

Firebase Realtime Database: A cloud-hosted NoSQL database that syncs data in real time across connected devices.

Front-end: The part of an application that users interact with directly, including the layout, design, and interactive elements.

Git: A version control system that tracks changes in code, allowing developers to collaborate, revert to previous versions, and manage project history efficiently.

iOS (iPhone Operating System): Apple's mobile operating system.

JSON (JavaScript Object Notation): Format for exchanging structured data.

Machine Learning (ML): A branch of artificial intelligence that enables applications to recognize patterns, make predictions, and improve over time by learning from data.

PyTorch: An open-source deep learning framework developed by Meta that provides flexibility for building and training ML models

Restful API: A standardized approach for designing web APIs that use HTTP requests (GET, POST, PUT, DELETE) to interact with resources

Swift: Apple's programming language for apps.

UI (User Interface): The visual and interactive elements of an application that users engage with.

Xcode: Apple's tool for building iOS and macOS apps.

YOLO (You Only Look Once): YOLO is a real-time object detection algorithm designed to identify and locate objects in images or video streams in a single pass through a neural network.

Introduction

Background

Honeywell's maintenance and part intake facilities require very thorough quality control to inspect their physical parts for issues like erosion, corrosion, and or damage. Currently, this process relies on a vocal guidance system and manual documentation, which are tedious and time-intensive. Our project aims to streamline these inspections using AI and image processing, enabling users to quickly capture, using a mobile device, part images and videos to assess their quality and or damage taken.

Document Summary

The System Architecture section displays a high-level view of how the major components of our system - the Swift User Interface, Application Logic, PyTorch ML, and Firebase Database - interact with each other both statically and dynamically. In the static system architecture diagram, the Swift User Interface initiates application logic which sends user data to the Firebase and part data to PyTorch ML where its dataset in Firebase is accessed. In the dynamic system architecture diagram, the relationships and interactions between each entity is shown where the User and Interface are tightly coupled, and the Database and ML Library entities are tightly coupled.

The Data Storage Design section portrays the ways we send and store user data using SwiftUI and Firebase. Furthermore, this section displays how data is sent and stored from PyTorch ML to firebase.

The Component Design section provides an overview of the static and dynamic relationships and interactions of each component in our application. Entities and classes that forms each component are displayed in the static and dynamic diagrams of the System Architecture section.

The UI Design section showcases the different interface screens of our IOS Intake Scanning App for both managers and employees, comprising of a welcome screen, login screen, home screen, setting screen, part data screen, management screen, and scanning screen.

System Architecture

Introduction

To provide a clearer understanding of our application's scanning process, the following sections present the static and dynamic architectural designs of our system. The static architecture diagram illustrates how the system is structured, showing the organization and interaction of components that form a unified framework. On the other hand, the dynamic architecture diagram demonstrates how our system operates by walking through an example of an intake part being processed by our application.

Rationale

Our architectural choices were guided by the need for scalability, ease of development, platform compatibility, and secure handling of user data. We divided our application into three main layers: Presentation, Business, and Database.

For the presentation, we chose Swift and SwiftUI to build a native iOS application, which provides smooth performance and a seamless user interface. Since our target platform was iOS, this choice offered the best user experience and integration with device features like the camera. We implemented a Python Flask server in the Business layer to handle data processing and model inference. Flask was chosen for its lightweight structure and ease of integration with PyTorch, which we use to implement and run our machine learning model. The Flask server receives scanned media from the app and returns a classification result. For database and user management, we selected Firebase, which offers built-in support for user authentication, a secure real-time database, and storage function. Firebase Auth ensures that only registered users can access or submit scan data, and all communications between the mobile app and Firebase occur over HTTPS, protecting data in transit.

From a security standpoint, Firebase's role was essential. We designed our system to restrict database access based on user authentication tokens, reducing the risk of unauthorized data modification or leakage. Using Firebase also eliminates the need to manually manage encryption or authentication logic. This security model influenced our decision to delegate data access and user identity management entirely to Firebase.

By choosing components that suited the platform's constraints and focusing on secure communication and access control, we made sure the system met its functional needs, safeguarded user data, and maintained overall reliability.

Static System Architecture

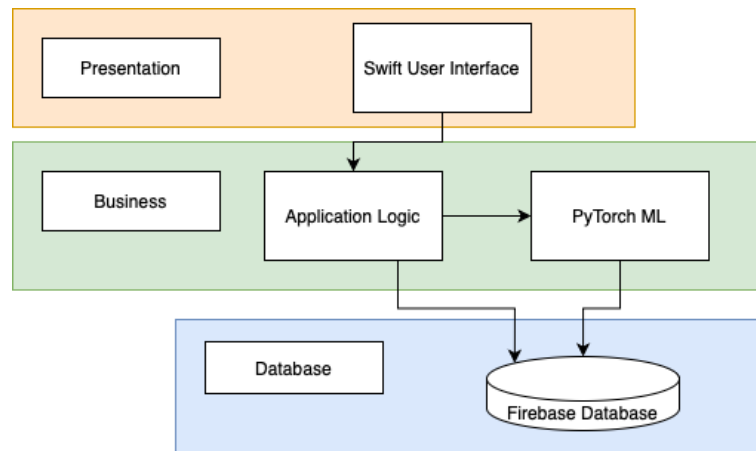


Figure 1 Static System Architecture

As our client did not give any strict architectural requirements besides that it needs to be an iOS application, we chose the components we are utilizing now to best accomplish the purpose of our project. This full application can be reduced to three major components. Our user interface is conducted through an iOS application made with Swift. The backend is composed of Flask for our framework and Firebase for the database. Most importantly, the machine learning model will be trained and tested with PyTorch.

The presentation layer consists of the Swift UI, which serves as the intuitive front-end of our application. This interacts with the application logic in our business layer, which processes the scans and sends it to our ML model. The model receives the data, processes it, and then returns if it's a valid or invalid scan. The information for these scans as well as authentication is stored in the Firebase Database, which is necessary as users may need to view this data to fulfill their tasks.

Dynamic System Architecture

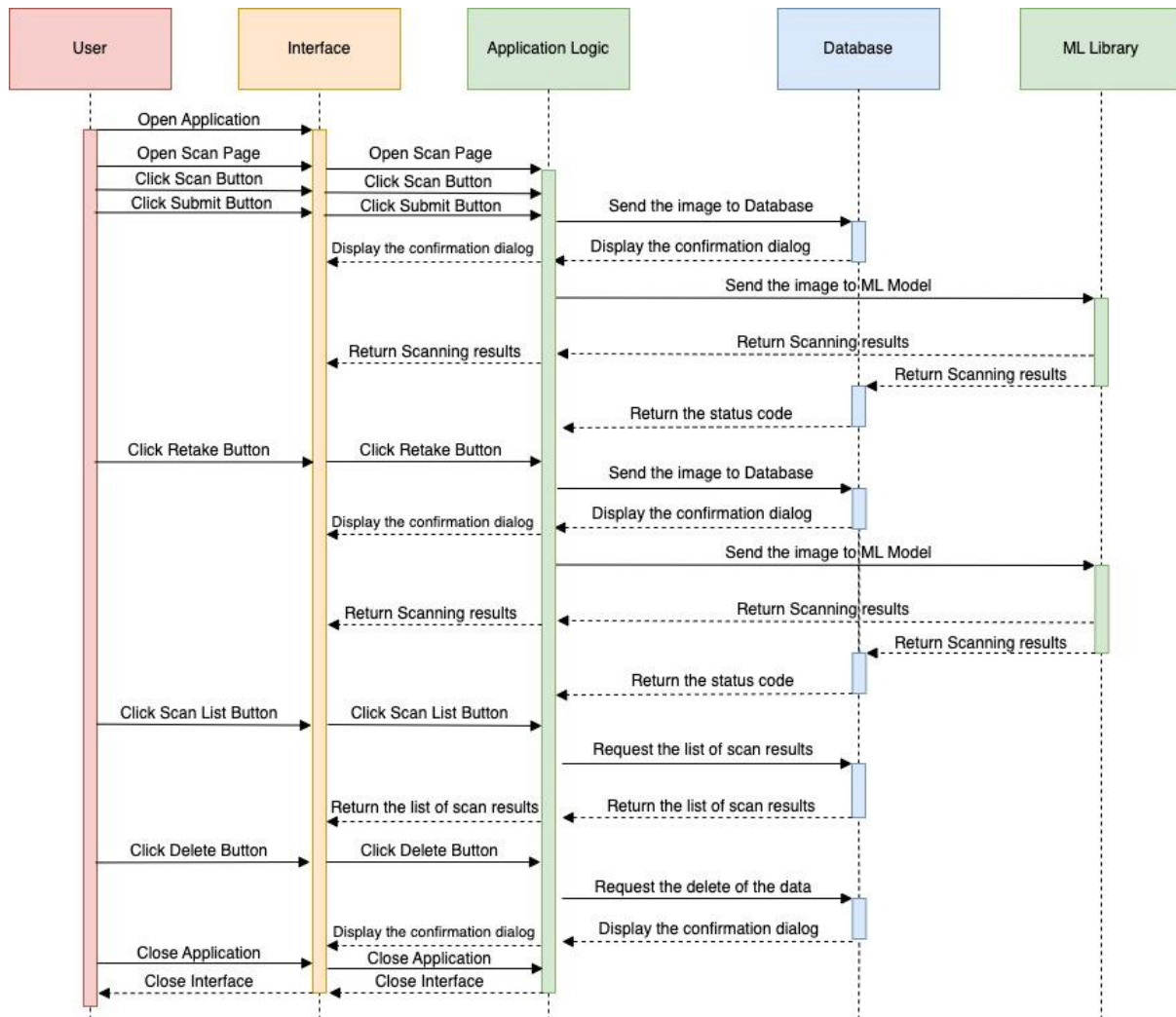


Figure 2 Dynamic System Architecture

The system sequence diagram (SSD) in Figure 2 represents the control flow of the main feature, which is scanning the object, storing the image/video in the database, and getting the result from the machine learning library. All our user inputs are processed via Interface and Application Logic. After the user opened the application and successfully logged in, the user should go to the scan page to scan the object. Once the user clicks the scan button, then the user will be able to see the scanned image. If the user likes the image and clicks the submit button, then the image will be sent to the database and machine learning model. After the image is successfully stored in the database and the machine learning

model, the confirmation dialog and the scanning results will pop up. The result will be updated in the database as well.

If the user wants to retake the picture/video, it will allow the user to scan the object again. After finishing scanning, the user can see the scanning result by clicking the scan list button. The scene will show the list of scans and if you click one of them, it will display the image, the employee's name, the scan date, and the scan result. Once the user wants to delete the data, the user has to click the delete button, and when it is clicked, the data with this scan will be deleted from the database and the confirmation dialog will be popped up.

Component Design

Introduction

The diagrams below display a static view of the Part Intake Scanning application's system structure and the dynamic behavior of the application's scanning and storing features. To depict a static overview of the application's components, a class diagram (Figure 3) was created, presenting the relationships between each class in a multilevel structure such as the StatsViewController which utilizes the Firebase API to retrieve part data. In order to exemplify the dynamic behavior of the application, a system sequence diagram was incorporated (Figure 4), demonstrating the interactions, scenarios, and functions between objects in the system such as the LoginViewController object having a function to store personal information to the Firebase object. Furthermore, the system sequence diagram gives an in-depth description on how the frontend and backend components will interact and influence each other. Together, these diagrams reinforce conceptual integrity by maintaining consistency between the system's static architecture and its dynamic behavior. Each runtime interaction shown in the sequence diagram directly corresponds to relationships and responsibilities defined in the class diagram, ensuring a unified and coherent design. Each diagram is crucial to understanding the application's design and functionality, providing a complete overview of the system's structure and behavior.

Static

Figure 3, below, demonstrates the static structure of our system, organized into three layers: Mobile Device View, Application Logic, and Data. Further, these layers are color coordinated to match the above System Architecture Diagrams. Also, controllers such as HomeViewController, ScanViewController, EmployeeViewController, StatsViewController, and SettingsViewController manage UI interactions and user input, which use components like buttons, labels, and text fields. Also, these controllers interact with the Application Logic layer, which includes the Server (a Flask-based backend using a YOLO model) and the Firebase API (handling authentication, real-time database operations). The Data layer contains Manager and Employee entities, with employees managing lists of Image and Video objects that include metadata such as classification, confidence scores, and timestamps. <<create>> arrows indicate the generation of these media objects, while <<use>> arrows highlight dependencies between components, such as how the ScanViewController interacts with the Server and Firebase API for processing scanned images and videos.

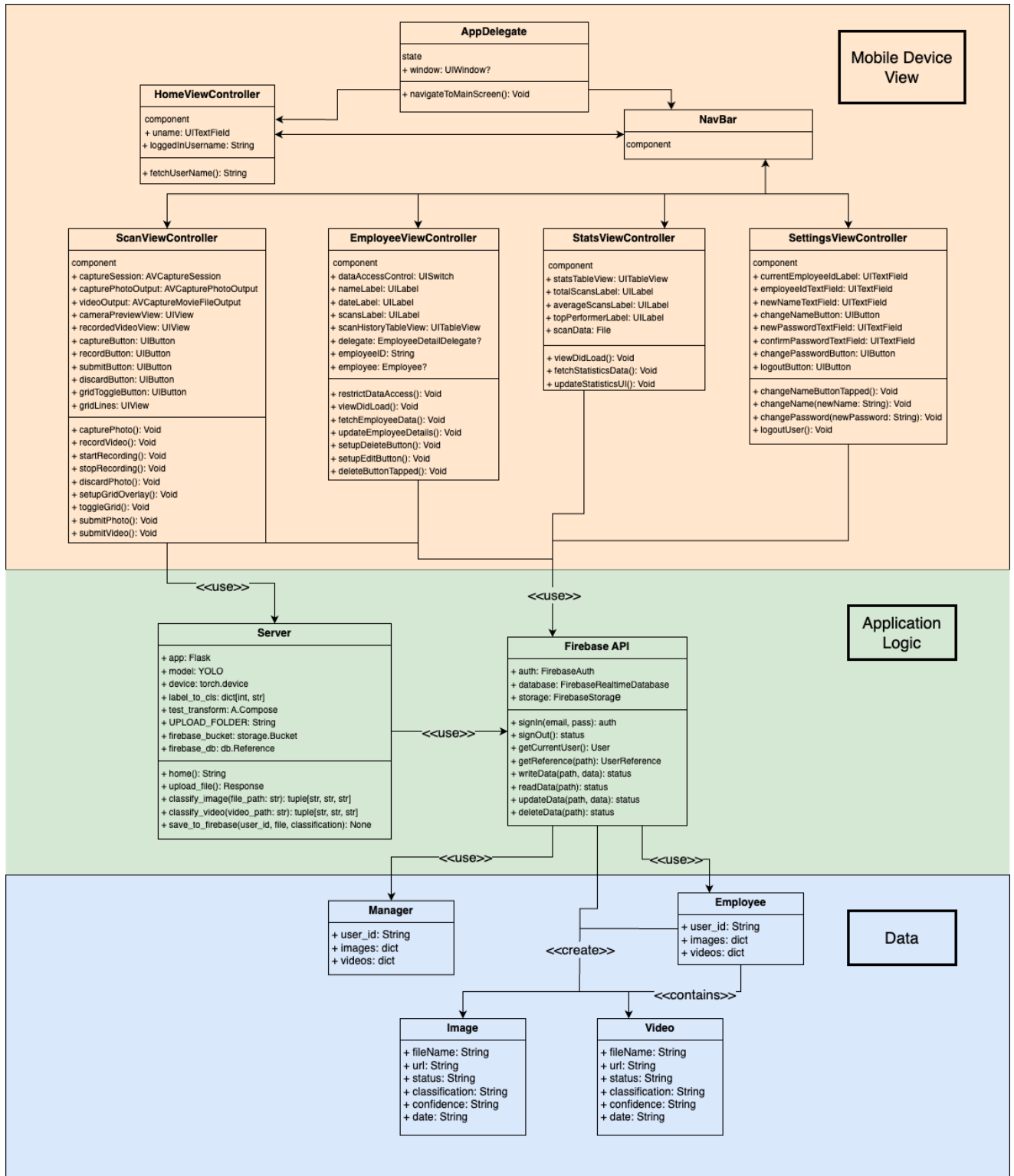


Figure 3 Component Design Class Diagram

Dynamic

Figure 4 shows the dynamic structures of the application. It focuses on the main feature of our project which is how scanning works on the device. All components are matched to the Component Design Static part and color coordinated as well. The diagram was designed under the assumption that the user is logged in to the application already. It shows which scenarios are the main feature of this project and which files are interacted in the scenario. HomeViewController is launched when the user successfully logs in to the application. ScanViewController file is used when the user tries to take a photo/video. This file is also used when trying to retake the photo/video that was not submitted yet. After scanning, the user can submit it to the database by sending it to the backend (Server), getting the scan results, and storing the integrated information(including the photo/video, and scan result) in the database(Firebase). Once it is successfully submitted to the database, the user can see the stored data, and ScanListViewController and ScanListImageViewController are used to fetch the data. For the same object, the user can see all scanning history regarding the object. The file StatsViewController is used in this action, and it retrieves the object data from the database, organizes the data by the object, and displays the list of them.

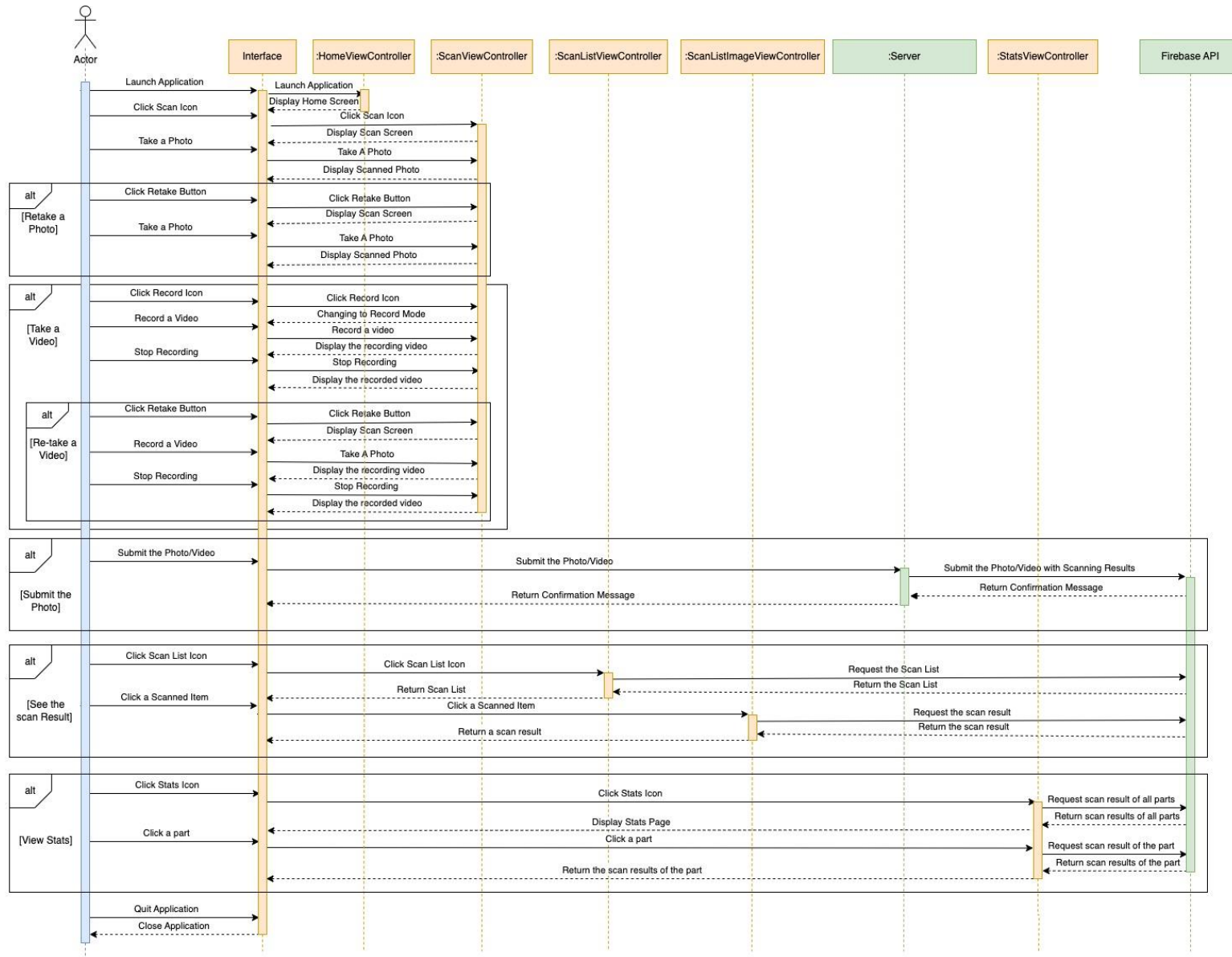


Figure 4 Component Design Dynamic Diagram

Data Design

Introduction

Figure 5 illustrates how data fields and values are stored in the Firebase real-time database, while Figure 6 provides an example of how employee data is specifically organized within Firebase. This storage method follows the NoSQL format of a document database. The document database is structured into collections, which contain partitions and nested entities or fields. These fields contain different typed values. Currently, the Firebase database only uses strings and integers to store user and activity log data. The values and fields are automatically indexed in this document database.

Database Use

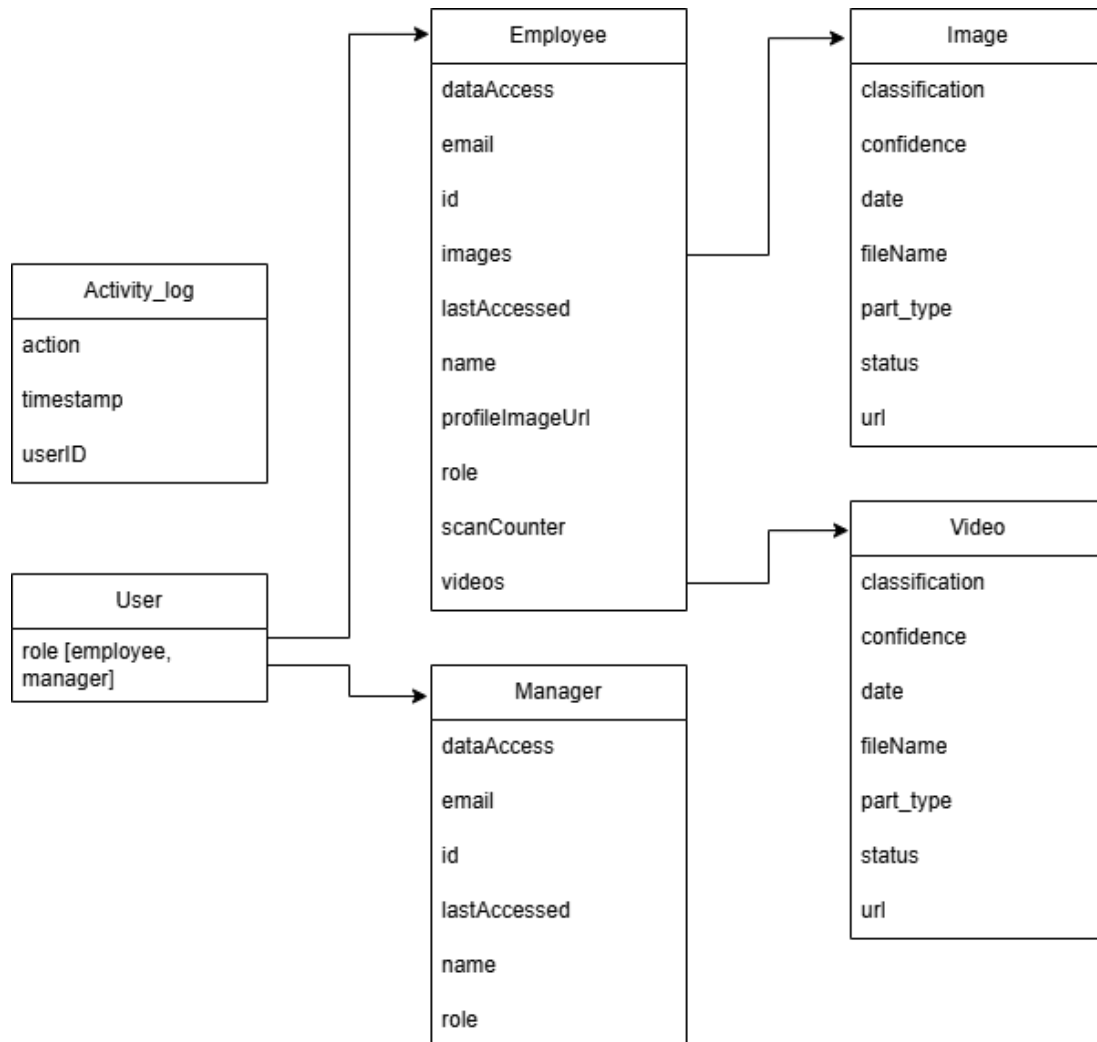


Figure 5 Document Database Diagram

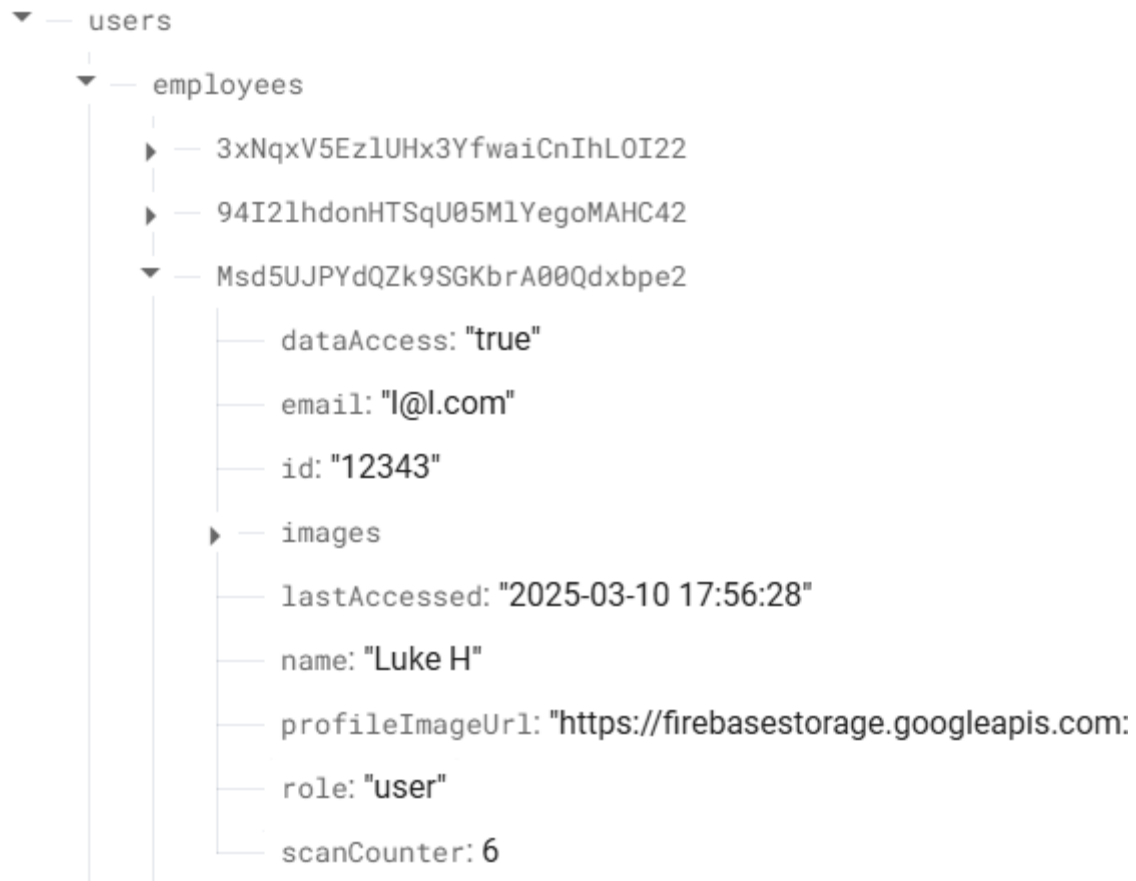


Figure 6 Document Database Example

File Use

Once the employee scans the item and clicks the submit button, the scanned image (.jpg) or video (.mov) is sent to the backend server. The backend then forwards this scan to the ML model, and the result returned by the model, along with the scanned file (.jpg or .mov) are subsequently sent to the database. When a manager attempts to export a user's data, the app generates a .csv file containing all user data and scan history/results. The manager can download this file for reference or reporting purposes, such as employee performance reports or item defect reports. Users can upload a profile image via the app, which is then stored into the database. Once uploaded, the image is displayed whenever the user accesses the Settings section. From the manager's perspective, this image is displayed in the Employees tab, allowing for easy recognition and differentiation of employees.

Data Exchange

Our solution exchanges data and information between devices through a private server and Firebase. The machine learning results from the iOS application are first sent to a private server over HTTPS. Once processed, these results are then sent to our Firebase Storage via a POST request, where they are securely stored. Also, we use Firebase Authentication on the app for user login and access management. Both Firebase and our private server rely on the use of authentication tokens to secure data exchanges and verify user identity.

Security considerations:

Given the nature of this application and dealing with sensitive company information (material scans and employee details), various security measures were accounted for and implemented. The data and information being sent back and forth between the iOS application, the private server, and Firebase is encrypted using HTTPS (TLS/SSL) protocols, which makes sure all user data such as passwords and emails are safely sent and stored. Plus, Firebase hashes and salts passwords so they aren't directly stored either. As well, authentication tokens are used to validate user sessions and prevent unauthorized access, with each request requiring a valid token to interact with the server or database. Firebase also uses rules-based access control, so, again, only authenticated users with the proper permissions can read or write specific data. There is no sensitive data being locally stored on our application other than cached images from scanning that get deleted once the app is closed out. Once a log has been taken and stored in our database, it does not get moved around unless the user decides to delete it. The only concern to keep an eye out for is user login data which needs to be input and verified through our backend and database. While this doesn't pose an inherent risk as passwords are hidden when entered, it could lead to exposed data if a user isn't careful when logging in.

UI Design

Introduction

In this section, we describe the user interface of our application. Users can log in either as an employee or a manager, with each role having access to different menu options. Employees can scan objects, view their scan list, check scan statistics, and update their username and password. Managers have additional capabilities: they can view employees' scan histories, manage user accounts, export scan data, access scan statistics, and update their own username and password.

Descriptions

When a user successfully logs into the application, they are greeted with the home screen. For employees, the home screen displays a personalized welcome message with the user's name (Figure 7). For managers, it shows the activity history of all employees (Figure 8).

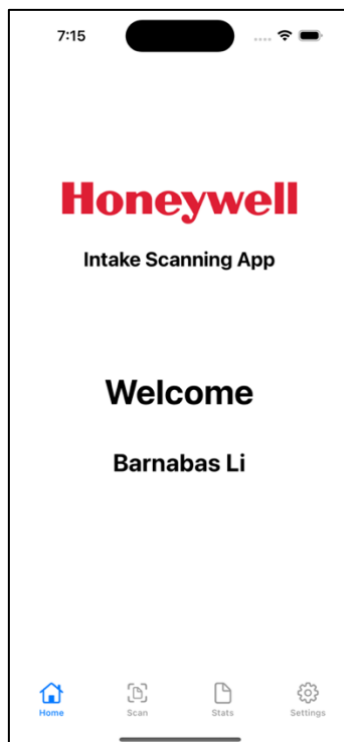


Figure 7 Home Screen (Employee)

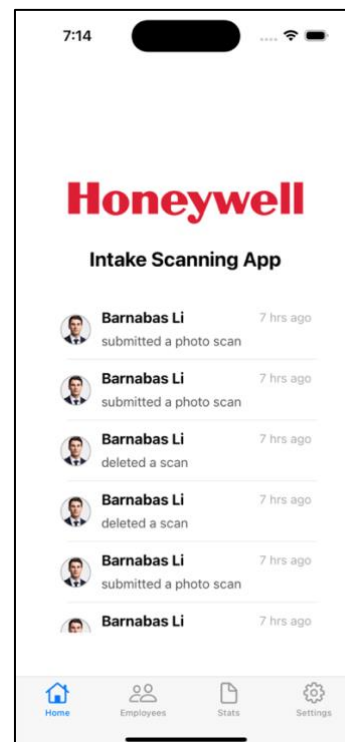


Figure 8 Home Screen (Manager)

Heuristics Evaluation: As soon as users log in, they get instant feedback. Employees see a personal welcome message, while managers get an activity feed (**visibility of system status**). The screen also stays focused, showing only what's needed, just a name or a

single feed to keep things simple (**aesthetic and minimalist design**). As well, navigation out of this screen is consistent since the tab bar stays the same across all other screens, so users always know how to get around our interface (**recognition rather than recall**).

The employee is able to navigate to the scan tab at the bottom of the screen and submit or retake either a video or photo using the camera button or record button as shown in figure 9. There is also a button to toggle grid lines in order to assist in photo/video alignment. After a photo or video is taken, the employee can identify the type of part present in the photo or video and press the submit button. Further, an alert will notify the employee that the photo or video has been submitted.

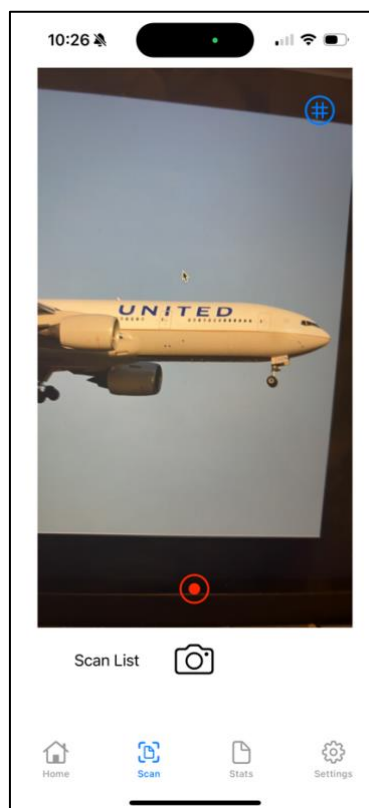


Figure 9 Scan Screen (Employee)

Heuristics Evaluation: The scan screen allows employees to take photos or videos using clearly labeled buttons that follow platform standards (**consistency and standards**). Also, users have more control by being able to retake photos/videos or toggling grid lines for alignment help (**user control and freedom**). Further, the alert that confirms the submission being successful, keeps users aware of the system's status (**visibility of system status**).

The employee is able to press the scan list button from the previous screen which displays each part photo and video the employee has taken as seen in figure 10. There is also a filter selection at the top of the screen to filter each part by date and part type and a refresh button to reset each filter. The employee can click on a part in the scan list which presents a screen displaying a color-coded response to whether the part is good or bad, the part type, confidence level, date, and a delete button to delete the scan as seen in figure 11.

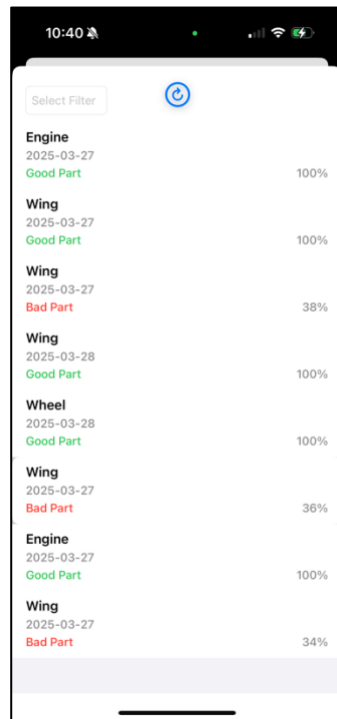


Figure 10 Scan List



Figure 11 Scan List Bad Part Example

Heuristics Evaluation: The scan list screen allows employees to filter submissions by date or part type and reset filters with a single tap, which gives them control over their view and interaction (**user control and freedom**). When viewing part details, the color-coded good/bad status and confidence indicators present key information while also keeping non-essential elements minimal (**aesthetic and minimalist design**). The confirmation pop-up and step before deletions also effectively prevents user errors (**error prevention**).

The employee can navigate to the stats screen through the tab bar at the bottom of the screen which shows the statistics of part type such as defect rate, part distribution, and scan history from every part the employee has submitted through different graphs and visualizations as seen in figure 12. If an employee chooses to click on a part from the stats screen, information such as failure rate over time, confidence rate over time, and defect distribution will appear as seen in figure 13.

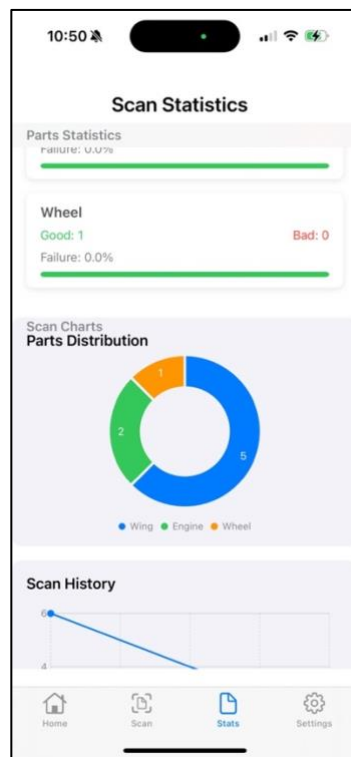


Figure 12 Scan Screen (Employee)

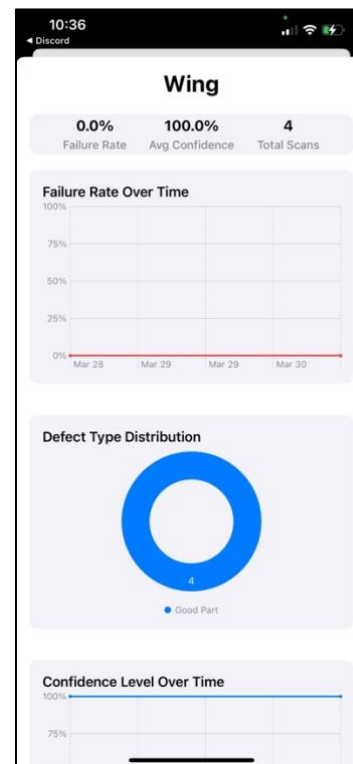


Figure 13 Part Statistics Example (Employee)

Heuristics Evaluation: Information displayed in the scan statistics screen are layered in easy-to-read and color-labeled charts with clear labels (**aesthetic and minimalist design**). When an employee desires to view a specific part's information, three charts are instantly created to display failure rate, defect type distribution, and confidence level (**visibility of system status**). If a user accidentally clicks on the wrong part to view, the user can simply swipe down on the screen to exit the pop-up and click on the correct part (**user control and freedom**).

For a manager, instead of a scan tab, there exists an employee tab where managers can view a list of each employee as seen in figure 14. A manager can click on an employee cell and view their name, date in which they last accessed their account, total scans, scan history, be able to restrict an employee's data access, be able to export as specific scan or all scans of an employee as a pdf or csv file and be able to edit an employee's details or delete the employee's account altogether as seen in figure 15.

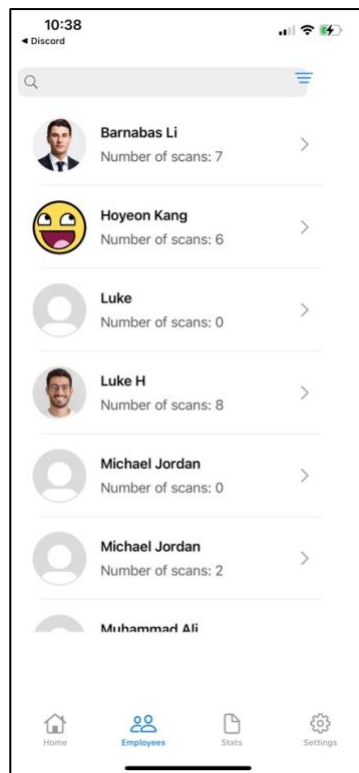


Figure 14 Employee Screen

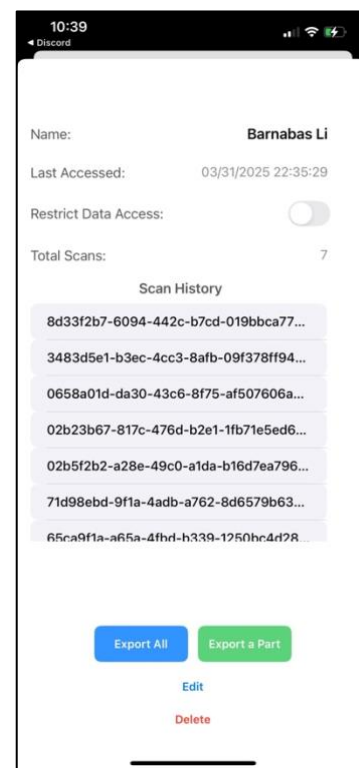


Figure 15 Employee Data Example

Heuristic Evaluation: The application sends a confirmation message to the manager if the manager decides to delete an employee's data, preventing any accidental deletions (**error prevention**). The number of scans for each employee is also displayed when an employee cell is clicked, decreasing a user's need to memorize information (**recognition rather than recall**). The only information listed under each employee cell in the employee list is their name, number of scans, and profile picture in a neat scrollable list (**aesthetic and minimalist design**).

The stats screen for the manager side displays each part type and the statistics of all parts such as number of good and bad parts and failure rate as shown by figure 16. When a part is long clicked, it displays more specific part stats similar to that of figure 13. The stats screen also allows the manager to compare the failure rate and confidence rate of two parts through different charts as seen in figure 17.

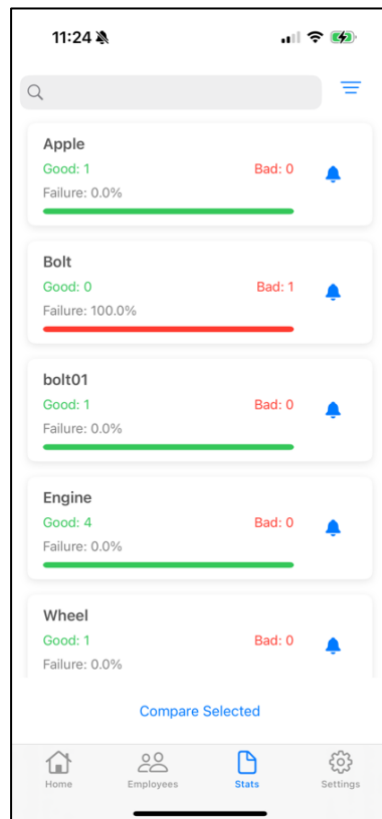


Figure 16 Stats Screen (Manager)



Figure 17 Part Stats Comparison Example

Heuristics Evaluation: When the manager selects a part, display charts are instantly created, matching and presenting data such as defect type distribution (**visibility of system status**). The labels above each chart are clear and understandable, mapping to real-world machine part descriptions (**match between the system and real world**). To undo a comparison between two parts, users can easily swipe down the pop-up and unselect the two parts (**user control and freedom**).

Appendix A

Team Information

Luke Hammond

Contact: lhammond9@gatech.edu

Contributions: Team leader, Primary Communicator, Backend and Frontend Development.

Hoyeon Kang

Contact: hkang326@gatech.edu

Contributions: UI Design, Backend and Frontend Development, Database Management

Barnabas Li

Contact: bli457@gatech.edu

Contributions: UI Design, Backend and Frontend Development, Documentation.

Christian Chin

Contact: cchin45@gatech.edu

Contributions: UI Design, Backend and Frontend Development, Documentation.

Nhat Nguyen

Contact: nnguyen354@gatech.edu

Contributions: UI Design, Backend and Frontend Development, Documentation.